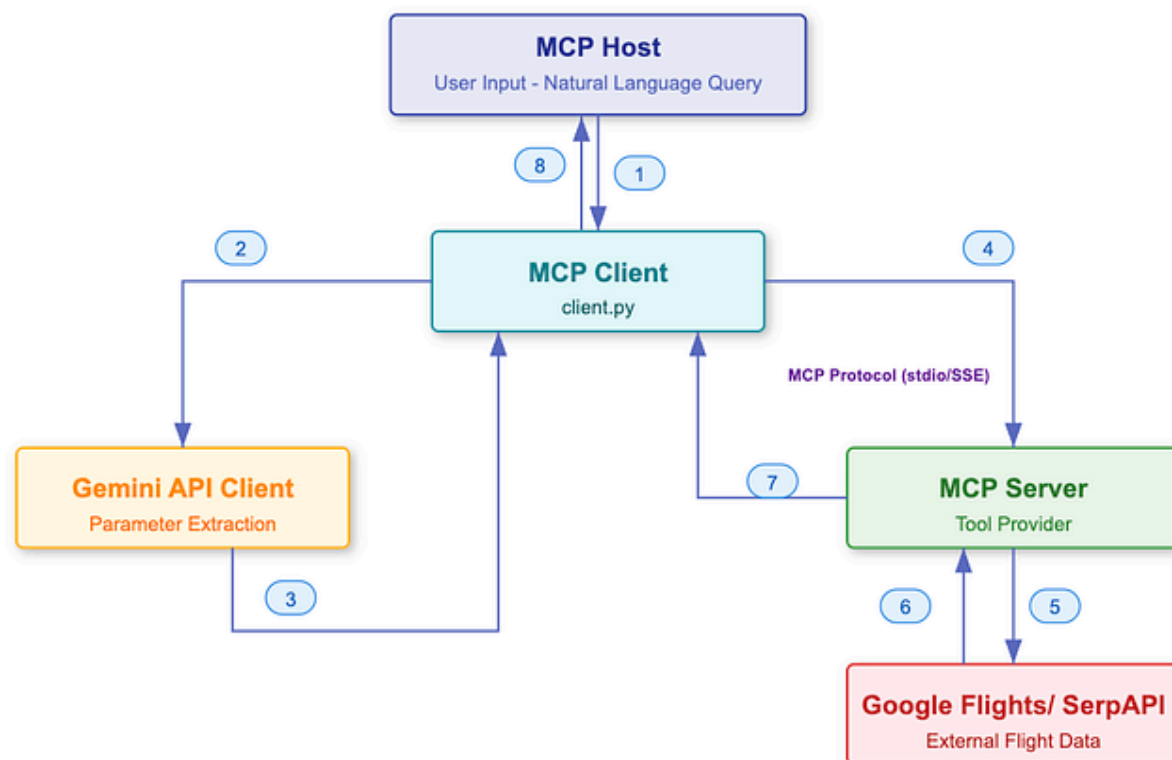


[< Go to the original](#)

Model Context Protocol (MCP) with Gemini LLM



Model Context Protocol(MCP) with Google Gemini LLM—A Deep Dive (Full Code)

**Arjun Prabhulal**

Follow

 Google Cloud - Community a11y-light ~10 min read · April 4, 2025 (Updated: April 7, 2025) ·

Free: Yes

As Large Language Models (LLMs) like GPT-4, Claude, Gemini and Llama3 evolve , we need standardized ways to connect them to tools, APIs, and systems .However, these models operate in isolation based on pre-trained data and don't have built-in access to real-time data, databases , external APIs, or local files.

In [my previous article](#) on **Function Calling**, I explained how it allows models to translate natural language into real-world actions and data access.While **Function Calling** provides the core capability, the **Model Context Protocol (MCP)** takes it a step further — making the entire process structured, flexible, and interoperable across a wide range of tools and systems.

In this article , we'll discuss more on **Model Context Protocol (MCP)** concept before diving into implementation :

1. What is Model Context Protocol (MCP) ?

sources and tools, acting as a universal connector for AI integrations.

Think of MCP as a "USB-C for AI integrations," providing a universal way for AI models to connect to different devices and data sources

How MCP works ?

MCP follows a **client-server architecture**, where:

- **Clients** (like AI applications or LLMs) connect to
- **Servers** (MCP tool providers) expose tools, APIs, or data sources to clients.

This enables dynamic and structured interactions between LLM models and the external API's.

Benefits of MCP:

1. **Standardized Integration:** Connect LLMs to any external system with less custom work.
2. **Flexibility:** LLMs can use multiple tools and services — on demand.

4. **Simplified Development:** Build and expose custom MCP servers easily.

5. **Easier Maintenance:** No more repetitive integration logic.

MCP transforms the API into a **model-friendly tool**, complete with auto-discovery, a predictable schema, and structured interaction.

Examples of MCP Servers:

- **File System:** Accessing local files and directories.
- **Web Search:** Run real-time web searches.
- **Databases:** Query SQL or NoSQL databases
- **CRMs:** Connecting to CRM systems like Salesforce.
- **Version Control:** Accessing version control systems like Git

When to use Model Context Protocol (MCP) ?

MCP to be used when:

- We're building **agentic systems**
- We want tools to be **modular, reusable, discoverable**

• we want scaling to multiple tools or toolchains

Architecture

Below project integrates multiple components to enable natural language flight search using Gemini + MCP

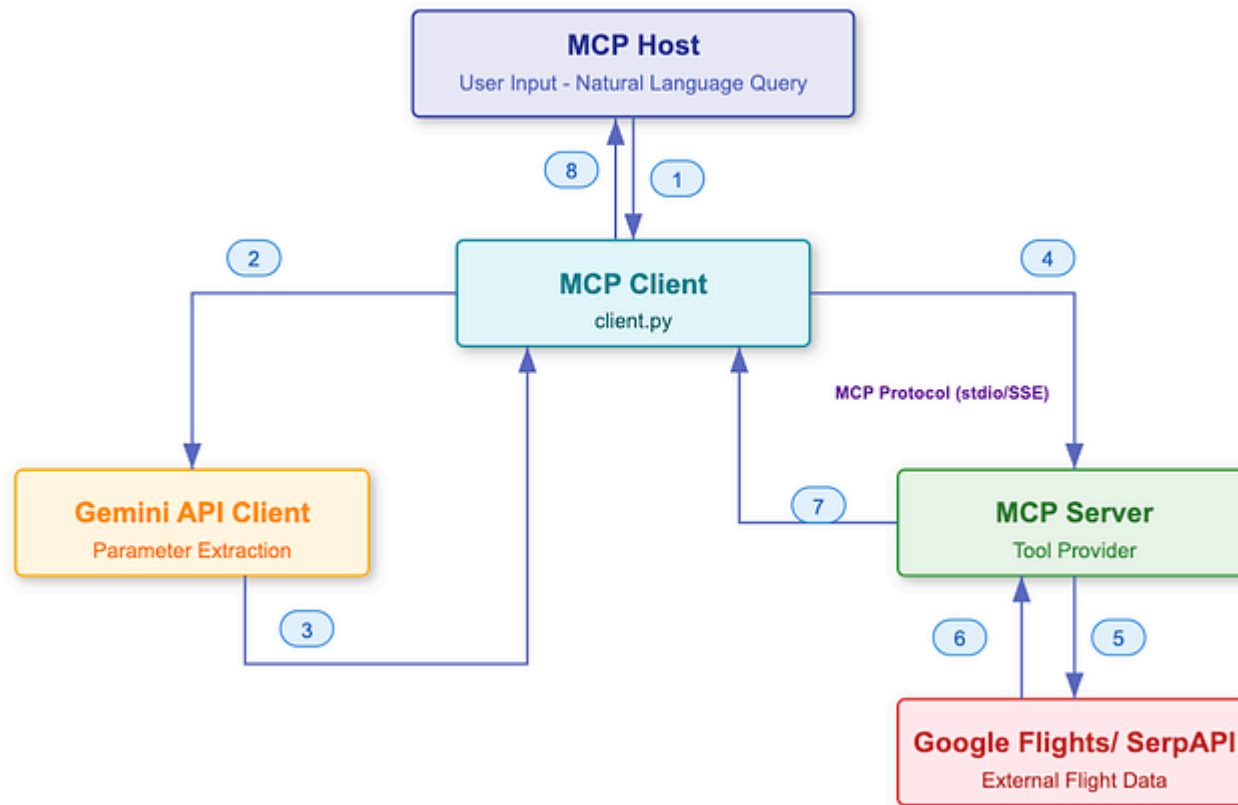


Image by Author

Component Interactions

1. User to Client

- Client script (``client.py``) processes the input

2. Client to MCP Server

- Client starts the MCP server process (``mcp-flight-search``)
- Establishes stdio communication channel
- Retrieves available tools and their descriptions

3. Client to Gemini API

- Sends the user's query
- Provides tool descriptions for function calling
- Receives structured function call with extracted parameters

4. Client to MCP Tool

- Takes function call parameters from Gemini
- Calls appropriate MCP tool with parameters
- Handles response processing

-
- MCP server makes requests to SerpAPI
 - Queries Google Flights data
 - Processes and formats flight information

Implementation

Let us dive into building this pipeline with Gemini AI by breaking down into key implementation steps

Pre-Requisites

1. Python 3.8+ installed
2. Google Gemini Generative AI access via API key
3. A valid SerpAPI key (used to fetch live flight data)

Step 1 : Setup virtual environment

Install the dependancies


```
#Setup virtual env
python -m venv venv

#Activate venv
source venv/bin/activate

#Install dependancies
pip install google-genai mcp
```

- **google-genai** : The official Python library for interacting with Google's Generative AI models (like Gemini).
- **mcp** : A Python SDK for interacting with an MCP (Model Context Protocol) server. This SDK likely provides functionalities to communicate with external tools or services.

Set Environment variables

Copy

```
export GEMINI_API_KEY="your-google-api-key"
export SERP_API_KEY="your-serpapi-key"
```

Step 2: Install the MCP Server — mcp-flight-search

For this article, we'll use [mcp-flight-search](#) — a lightweight MCP Server built using [FastMCP](#) which exposes a tool that searches real-time flight data using the SerpAPI.

Install MCP server package where I published to PyPi

<https://pypi.org/project/mcp-flight-search/>

Copy

```
# Install from PyPI
pip install mcp-flight-search
```

let us verify if MCP server package is installed successfully

```
> pip show mcp-flight-search
Name: mcp-flight-search
Version: 0.2.0
Summary: Flight search service implementing Model Context Protocol (MCP) tools
Home-page: https://github.com/arjunprabhulal/mcp-flight-search
Author:
Author-email: Arjun Prabhulal <code.aicloudlab@gmail.com>
License:
Location: /Users/arjunprabhulal/ai-projects/mcp-gemini-flighsearch/venv/lib/python3.11/site-packages
Requires: fastmcp, google-search-results, pydantic, python-dotenv, rich
Required-by:
```

Step 3 : Understanding MCP Tool Packages

Import library which initializes both **Gemini** and **MCP** SDKs and prepares for async execution.

Copy

```
from google import genai
```

Above imports the **genai** module from the google-generativeai library. It provides access to **Google's powerful LLMs**, such as Gemini 1.5 and 2.0,2.5 model and includes client methods to interact with models using natural language.

Copy

```
from google.genai import types
```

Above module gives access to the **type definitions and configuration structures** used by the Gemini API. For example:

- **Tool:** Defines tools (functions) that the model can call.

Copy

```
from mcp import ClientSession, StdioServerParameters
```

Above classes come from the [mcp-sdk-python](#) library and are essential for interacting with **MCP servers**:

- **ClientSession**: Manages the communication session between our client/app and the MCP server.
- **StdioServerParameters**: stdio allows the server to be language-neutral and easily embedded in different environments.

Copy

```
from mcp.client.stdio import stdio_client
```

This imports the **stdio_client**, an **asynchronous context manager** used to establish a connection with an MCP server over standard I/O. It ensures that the server is correctly launched, and the client is ready to send/receive structured requests.

```
from google import genai
from google.genai import types
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
```

Above 4 key imports together form the backbone of how we bridge **Gemini's LLM** interaction with **real-world APIs** exposed via **MCP tools**.

Step 4: Initialize Gemini Client

Copy

```
client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))
```

genai.Client() is the primary interface used to interact with Google's generative models (e.g., Gemini 2.5 Pro, Gemini 2 Flash)

Once GenAI Client is initialized, this client object can:

- Send prompts to Gemini models
- Pass tool definitions (function calling)
- Receive structured responses and function call objects

Step 5 : Configure MCP Tool Server

Below block sets up the parameters required to **launch and communicate with the MCP server** that exposes tools (in our case, a flight search function).

Copy

```
server_params = StdioServerParameters(  
    command="mcp-flight-search",  
    args=["--connection_type", "stdio"],  
    env={"SERP_API_KEY": os.getenv("SERP_API_KEY")},  
)
```

mcp-flight-search — This is the CLI entry point to run local MCP server, or could be a Python module in our case that implements the **MCP protocol**.

stdio — This tells the server to use **standard input/output (stdio)** as its communication channel. Stdio is simple, language-agnostic, and great for running tool servers locally or in subprocesses.

SERP_API_KEY — This passes an environment variable (SERP_API_KEY) to the subprocess running the tool. In our case, the tool needs it to authenticate with **SerpAPI**, which fetches real-time flight data

stdio_client async context manager.

Copy

```
Python 3.11.11 (main, Dec 3 2024, 17:20:40) [Clang 16.0.0 (clang-1600.0.26.4)]
Type "help", "copyright", "credits" or "license" for more information.
>>> import os
>>> from google import genai
>>> from google.genai import types
>>> from mcp import ClientSession, StdioServerParameters
>>> from mcp.client.stdio import stdio_client
>>>
>>> client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))
>>>
>>> server_params = StdioServerParameters(
...     command="mcp-flight-search",
...     args=["--connection_type", "stdio"],
...     env={"SERP_API_KEY": os.getenv("SERP_API_KEY")},
... )
>>> server_params
StdioServerParameters(command='mcp-flight-search', args=['--connection_type',
>>>
```

The **Gemini client** handles language understanding, prompt generation, and function calling.

Step 6 : Connecting to MCP Server and listing tools

Below block of code does three important steps

1. Starts connection with the MCP server ,
2. Initializes a session for structured tool communication and
3. Dynamically discovers and formats available tools for Gemini.

Copy

```
async def run():
    # Remove debug prints
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            prompt = f"Find Flights from Atlanta to Las Vegas 2025-05-05"
            await session.initialize()
            # Remove debug prints

            mcp_tools = await session.list_tools()
            # Remove debug prints
            tools = [
                types.Tool(
                    function_declarations=[
```



```
        description = tool.description,
        "parameters": {
            k: v
            for k, v in tool.inputSchema.items()
            if k not in ["additionalProperties", "$schema"]
        },
    }

    )

    for tool in mcp_tools.tools
]

# Remove debug prints

response = client.models.generate_content(
    model="gemini-2.5-pro-exp-03-25",
    contents=prompt,
    config=types.GenerateContentConfig(
        temperature=0,
        tools=tools,
    ),
)
```

Let us breakdown line-by-line to understand how the MCP Client-Server communication is happening under the hood along with Gemini LLM

stdio_client is an asynchronous context manager that handles:

- managing the input/output streams for message exchange

read and write objects are asynchronous streams

- **read**: reads responses or tool registration from the server
- **write**: sends requests or tool invocations to the server

Copy

```
prompt = f"Find Flights from Atlanta to Las Vegas 2025-05-05"
```

Above prompt is **natural language query** we'll send to the Gemini model. which Gemini will later turn into a structured tool call.

Copy

```
await session.initialize()
```

session.initialize() is the one which triggers the initial **MCP handshake** between our client and the server.

server registers its available tools (in our case: a flight search tool).

- Session is now ready to list, call, and execute tools.

Copy

```
mcp_tools = await session.list_tools()
```

Above requests the list of all tools (functions) exposed by the server.

- Each tool in `mcp_tools.tools` contains:
- A name
- A description
- An input schema (i.e., what parameters it accepts, in JSON Schema format)

`mcp_tools.tools` makes the MCP server **self-describing**, so that LLM can automatically understand how to call each tool.

Copy

```
tools = [  
    types.Tool(  
        function_declarations=[  
            {  
                "name": tool.name,
```

```

        for k, v in tool.inputSchema.items()
        if k not in ["additionalProperties", "$schema"]
            },
        }
    ]
)
for tool in mcp_tools.tools
]

```

Above step converts the MCP tool definitions into Gemini's function_declarations format.

Now that our MCP server is running and session is initialized to discover tools from MCP Server for Gemini to use

Step 6 : Gemini — Interprets Prompt and suggest a Function Call

Copy

```

response = client.models.generate_content(
    model="gemini-2.5-pro-exp-03-25",
    contents=prompt,
    config=types.GenerateContentConfig(
        temperature=0,

```

Finally the user's prompt is sent to the Gemini model, along with a list of available tools discovered from the MCP server.

If Gemini recognizes the prompt as matching a function's schema, it returns a **function_call** object that includes the tool name and the auto-filled parameters.

Copy

```
result = await session.call_tool(  
    function_call.name, arguments=dict(function_call.args)  
)
```

Step 7 : Gemini LLM final response

If Gemini determines that the prompt aligns with a function (based on name, description, or parameters), it returns a structured function_call object like:

Copy

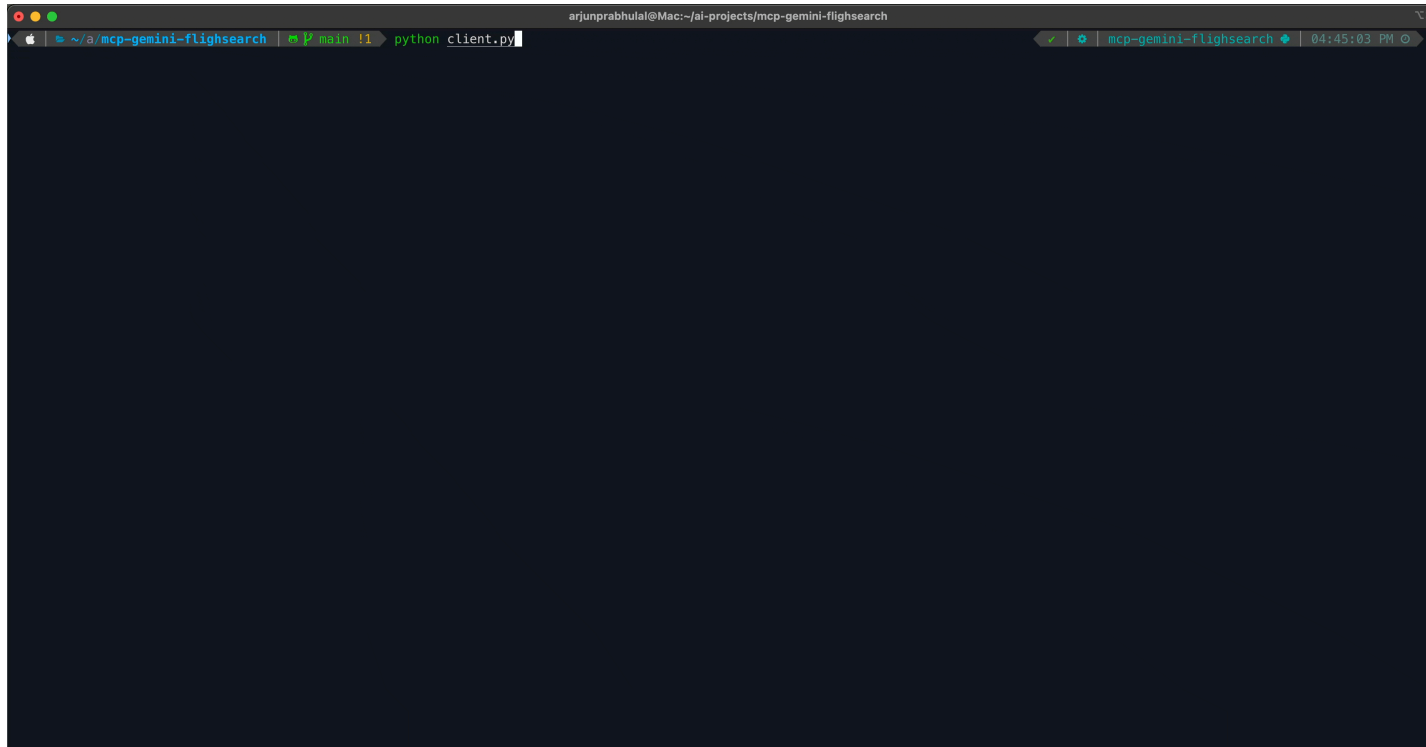
```
{  
  "function_call": {
```

```
    "source": "ATL",  
    "destination": "LAS",  
    "date": "2025-05-05"  
  }  
}  
}
```

Gemini LLM transitions from a passive text model to an **active decision-maker** that:

- Interprets natural input
- Selects an appropriate tool
- Fills in the function's arguments automatically
- We didn't write any parsing logic.
- Gemini LLM model filled in all the fields by interpreting the user's natural language.
- function call is structured and ready for execution.

Final Demo: Gemini 2.5 Pro with MCP



Best Practices for Using Model Context Protocol (MCP) with Gemini LLM

1. Tool Design

- **Clear Tool Names:** Use short, meaningful names (e.g., `search_flights`, `get_weather`).

- **Use Strong Typing:** Define input parameters explicitly (e.g., string, enum, number) to help the model fill them accurately.

2. Model Interaction

- **Fewer Tools means Better Accuracy:** Avoid overloading the model — stick to relevant tools only.
- **Dynamic Tool Loading:** Load tools based on the user's query or conversation context.
- **Prompt the Model Clearly:** Set the model's role and explain how and when to use the tools.
- **• Prompt the Model Clearly:** Set the model's role and explain how and when to use the tools.

3. Server Setup

- **Use stdio for Simplicity:** Start MCP servers using — connection_type stdio for easy local development.
- **Pass Environment Variables Safely:** Use env to send keys like SERP_API_KEY securely to tool servers.

- **Initialize Session First:** Always run `session.initialize()` before listing or calling tools.
- **List Tools Dynamically:** Use `session.list_tools()` to keep client flexible and tool-agnostic.

5. Error Handling & Security

- **Return Helpful Errors:** Make sure tool server responds with meaningful messages when something fails.
- **Secure APIs:** Never expose secrets like API keys in logs or error messages.

GitHub Repository:

You can access all the code used in this tutorial in my GitHub:

GitHub - arjunprabhulal/mcp-gemini-search: natural language flight search system using Google's...

natural language flight search system using Google's Gemini LLM with Model Context Protocol (MCP) . Convert...

github.com

Limitations

As of this writing(March 2025), there are a few limitations when using **Model Context Protocol (MCP)** and function calling with Gemini LLM:

1. Partial OpenAPI Support
2. Supported parameter types in Python are limited.
3. Automatic function calling is a Python SDK feature only.

Conclusion

In this article, we explored how to build a real-time, tool-augmented AI assistant using **Model Context Protocol (MCP)** with **Google Gemini LLM**. Through a step-by-step walkthrough, We've seen how:

- Gemini can process natural language prompts,
- MCP exposes structured, self-describing tools,
- MCP + Function calling enable seamless function execution powered by real-time data.

By integrating our AI application with external tools (in this demo like [mcp-flight-search](#)) with LLMs — Gemini 2.0 Flash or Gemini 2.5 Pro experimental

Freedium

ko-fi librepay patreon

without hardcoded logic

#google-gemini

#model-context-protocol

#ai

#generative-ai

#machine-learning